Qwen

From Prose to Queries: Constructing an Operational Database Layer for Lyapunov Stability and Lane Governance

Foundations of the Ecosafety Framework: Defining the Frozen Mathematical Invariants

The operationalization of safety within the EcoNet constellation is built upon a rigorously defined set of mathematical invariants and governance rules, often referred to as the "frozen math." These principles are not abstract theories but are specified as concrete formulas and constraints within the system's documentation, forming the unchangeable backbone of its safety calculus . The primary objective of this report is to project these frozen constructs into a normalized, queryable SQLite schema, thereby enabling agents across the constellation to assess safety states directly through structured queries rather than by reconstructing logic from prose descriptions or application code . This section deconstructs the foundational elements of this framework, establishing the precise definitions and relationships that will serve as the basis for the subsequent design of database tables and views. The core pillars of this framework are the Lyapunov residual, which quantifies system stability; the Knowledge-Ecological-Risk (KER) score, which governs lane transitions; the topology risk metric, which measures governance integrity; and the explicit corridor and blast-radius constraints that define safe operational envelopes. Understanding these components in their entirety is the first step toward creating a database layer that accurately reflects and enforces them.

The cornerstone of the entire safety model is the Lyapunov residual, denoted as V_t. It serves as the primary metric for the system's global stability, with the fundamental invariant being that any state transition must result in non-increasing residual energy, i.e., $V_{t+1} \leq V_t$. The formula for the Lyapunov residual at time t is explicitly defined as the weighted sum of squared risk coordinates across all relevant planes:

$$V_t = \sum_j w_j r_{t,j}^2$$

. Each term in this summation, $w_j r_{t,j}^2$, represents the contribution of a specific risk plane j to the total residual. The components of this formula are themselves well-defined research objects. The risk coordinate $r_{t,j}$ is a normalized value, constrained to the interval $[0,1]$, that signifies the severity of risk on plane j at time t . A value of 0 indicates a safe state, while a value approaching 1 signifies a high-risk condition. These coordinates are stored in tables like `shardriskcoord`, which provides the per-shard, per-plane risk values over time .

Complementing the risk coordinates are the plane weights, w_j. These are non-negative constants that establish the relative importance of each risk plane within the overall stability calculation . For instance, a critical ecological plane might be assigned a higher weight than a less critical materials plane. These weights are not arbitrary; they are governed by a formal contract known as the `PlaneWeightsShard`, which defines the parameters for a given `planecontractid` . The relationship between a shard instance and its governing weights is maintained through the `shardinstance` table, which stores the `planecontractid` associated with a particular shard . The `residualterm` table further refines this by defining coefficients, labeled `alpha`, for each combination of `kernelid` and `coordid`, allowing for flexible polynomial forms in the residual calculation . The existence of these distinct tables—`shardriskcoord`, `planeweights`, `residualkernel`, and `residualterm`—demonstrates a deliberate effort to create a modular and extensible framework for the residual's computation .

From the individual risk coordinates, three derived metrics are computed to facilitate governance and lane management: the Knowledge-Ecological Score (`K_t`), the Residual Energy (`E_t`), and the Residual Risk (`R_t`). The Residual Risk, R_t, is simply the maximum risk observed across all planes at a given time:

$$R_t = \max_j (r_{t,j})$$

. This captures the worst-case scenario on any single plane. Building on this, the Residual Energy, E_t, is defined as the complement of the residual risk:

$$E_t = 1 - R_t$$

. This metric incentivizes the reduction of the highest individual risk plane. Finally, the Knowledge-Ecological Score, K_t, is a separate, contract-bound value in the range $[0,1]$ that is derived from evidence richness and governance quality, typically sourced from a `knowledgeecoscore` table or function . Together, these three metrics—K_t, E_t, and

R_t—form the KER score, which acts as a compact representation of a shard's safety posture and is used extensively in lane governance decisions .

A critical aspect of the safety model is the concept of non-offsettable planes. Certain planes, such as those representing carbon impact or biodiversity, are designated as non-compensatory . This means that a degradation in one of these planes cannot be offset by improvements in others. To enforce this invariant, a guard function, `checksafestep`, is implemented in shared modules like `kerresidual` . This function consults a flag in the `planeweights` table to identify non-offsettable planes and will block any proposed state transition that results in a worsening of these critical coordinates, even if the overall Lyapunov residual improves . This ensures that fundamental ecological or safety guarantees are never compromised in pursuit of optimizing other, less critical metrics. This rule is a hard constraint embedded within the system's logic, making it a crucial element to encode in any queryable safety surface.

Another vital component integrated into the Lyapunov framework is the topology risk, denoted $r_{topology}$. This metric moves beyond internal shard health to measure governance drift and structural integrity within the EcoNet constellation itself. It is calculated based on an inconsistency index, $I_{topology}$, which is itself a weighted sum of detected issues, such as missing `.econet` manifests ($N_{missing}$) and mislabeled repositories ($N_{mislabel}$):

$$I_{\text{topology}} = w_{\text{missing}} N_{\text{missing}} + w_{\text{mislabel}} N_{\text{mislabel}}$$

. This raw index is then normalized into a coordinate, r_{topology}, which falls within the unit interval $[0,1]$. This normalized value is not treated as a separate metric but is instead injected directly back into the Lyapunov residual as an additional term:

$$V_t^{\text{final}} = V_t^{\text{core}} + w_{\text{topology}} r_{\text{topology}}^2$$

where V_t^{core} is the residual computed from other risk planes and w_{topology} is the specific weight assigned to the topology plane . By doing so, the system makes governance drift—a failure to maintain manifest completeness or role-labeling consistency—a first-class citizen in its stability calculus. Any deviation from the expected topology structure directly increases the Lyapunov residual, creating a powerful incentive for maintaining a healthy and well-governed network .

The ultimate purpose of calculating the Lyapunov residual and KER scores is to enforce lane governance rules. These rules dictate the permissible operational context for an artifact or shard, such as whether it can operate in a `RESEARCH`, `EXPPROD`, or `PROD` lane

. Each lane has a specific set of thresholds that a workload must satisfy. For example, promotion to a `PROD` lane requires meeting stringent gates such as $K_t \geq 0.90$, $E_t \geq 0.90$, and $R_t \leq 0.13$. Beyond simple threshold checks, lane progression is governed by several key invariants. First, the process must be monotonic; once a shard enters a higher lane, it cannot be rolled back to a lower one . Second, every transition must preserve Lyapunov descent, meaning the residual after the transition, V_{t+1}, must be less than or equal to the residual before, V_t . Third, the transition must pass a series of admissibility predicates, which include checks for corridor compliance, status of non-offsettable planes, and overall topology health . These rules are documented in artifacts like `LaneStatusShard` and enforced via verdict tables such as `virtalaneverdict` . Making these predicates queryable is a central goal of this research.

Finally, the safety framework incorporates explicit spatial and temporal constraints defined by blast-radius metrics and corridor boundaries. The blast radius of a node defines the envelope of potential impact from failures or anomalies, characterized by dimensions like physical distance in meters (`radiusmeters`), graph hop count (`radiushops`), and propagation time (`radiustimehours`) . These metrics are stored in tables like `blastradiusobject` and `nodeadjacency`, which together describe the adjacency graph of the network . The concept of a "canal" implies a path along which influence propagates, and its reach is determined by traversing the adjacency graph . The safe operational space for a shard is further constrained by corridors, which are multi-dimensional polytopes in risk space. These corridors are defined in the `corridordefinition` table, which binds each risk coordinate (`varid`) to safe, gold, and hard bands, along with weights . A shard is considered compliant with a corridor if its vector of risk coordinates, `x`, satisfies a set of linear inequalities, `Ax ≤ b` . These spatial and corridor-based constraints ensure that the placement and operation of shards are physically sensible and respect both immediate local hazards and broader systemic risk boundaries. The challenge lies in integrating all these disparate but interrelated frozen objects—the residual formula, KER scores, topology risk, lane rules, and spatial metrics—into a cohesive and directly queryable database architecture.

Identifying the Implementation Gap: From Theory to Machine-Queryable Surfaces

While the theoretical underpinnings of the ecosafety framework are robustly defined, a significant gap exists between this frozen theory and its practical implementation in the database. The core issue is not a lack of mathematical or logical primitives but a lack of

systematic projection of these constructs into a set of unified, machine-queryable surfaces . Currently, the logic for assessing safety states is fragmented across multiple tables and application-level code, forcing agents to perform complex joins and re-implement calculations manually . This fragmentation leads to potential inconsistencies, reduces transparency, and hinders automation. The user's directive to create specific views and tables—such as `vshardresidual`, `vshardtopologyker`, `vshardblast`, and `vlane_admissibility`—is a direct response to this gap, aiming to expose more of the underlying math as concrete, reusable database artifacts . This section analyzes the specific areas where the current implementation falls short and explains why the introduction of these new surfaces is necessary to achieve the research goal of making safety-critical objects directly queryable by agents.

One of the most prominent gaps is the absence of a canonical, shard-centric view for computing the Lyapunov residual, V_t. Although the necessary building blocks are present in the database—in tables like `shardriskcoord` (for $r_{t,j}$ values), `residualterm` (for α coefficients), and `planeweights` (for w_j)—there is no single, self-contained view that performs the aggregation $V_t = \sum_j w_j r_{t,j}^2$. As a result, any agent needing the residual for a given shard instance must construct this calculation using a complex SQL query that joins `shardinstance` with `shardriskcoord` and `residualterm`. This manual reconstruction of the formula is inefficient and error-prone. Furthermore, the full safety snapshot, which includes not just the residual but also the derived KER scores and the `lane` status, is not presented in a single place . An agent wishing to evaluate a shard's complete safety posture must join `shardinstance` with the results of the residual calculation, along with other tables for `knowledgeecoscore`, to derive K, E, and R. This piecemeal access to safety data impedes rapid decision-making and analysis. The need for views like `vshardresidual` and `vshardker` arises directly from this fragmentation, aiming to create a "single source of truth" for shard-level safety metrics that is both efficient to query and immune to manual calculation errors .

A second major area of concern is the incomplete integration of topology risk, $r_{topology}$, into the primary safety assessment surfaces. While the `topologyriskmetric` table computes the raw inconsistency index ($I_{topology}$) and the normalized coordinate ($r_{topology}$), and the `lane_status_shard` tracks flags like `topologyok`, these pieces of information are not consolidated with the core Lyapunov residual in a way that allows for straightforward querying . An analyst or automated system wanting to understand the total impact of governance drift on a shard's stability must currently write a multi-step query: first calculate the core residual, then fetch the `rtopology` value, and finally apply the weight and square it to get the additional penalty term. There is no view that presents the core residual, the raw topology index, the normalized topology coordinate, the topology weight, and the final residual including the topology penalty in a single row.

This lack of a unified view prevents simple, declarative questions like, "Show me all shards where the topology risk is causing the residual to exceed the hard band." The proposed `vshardtopologyker` view is designed specifically to close this gap by creating a comprehensive safety profile that treats topology risk as an inseparable part of the total Lyapunov function, making its contribution immediately visible and analyzable .

The third significant gap lies in the dispersion of spatial and temporal metrics related to blast radius and canal propagation. The blast radius of a node, which defines its sphere of influence, is described by multiple attributes (`radiusmeters`, `radiushops`, `radiustimehours`) spread across tables like `blastradiusobject` and `nodeadjacency` . Similarly, the concept of a "canal," which implies a directed path of influence propagation, relies on the adjacency graph defined in `adjacencygraph` . However, there is no consolidated view that provides a per-shard summary of its blast radius characteristics. An agent responsible for node placement, for instance, cannot easily ask for a list of all nodes whose blast radii overlap in a specific region. The logic for calculating derived metrics, such as the hypothetical "canal velocity" (r_{canal}), is also not materialized in the database . While the idea of deriving a normalized canal velocity from meters and hours is mentioned, there is no view like `vshardcanal` to compute and expose this as a first-class risk coordinate . This forces placement advisors and other agents to implement their own logic for resolving spatial relationships, leading to potential inconsistencies and duplicated effort. Creating `vshardblast` and `vshardcanal` would provide standardized, queryable surfaces for these spatial metrics, allowing agents to reason about physical proximity and propagation speed directly within SQL .

Fourth, the logic governing lane admissibility is largely implicit, residing in prose documents and Rust sketches rather than as a formal, queryable predicate in the database . The rules for promoting an artifact from `RESEARCH` to `EXPPROD` are complex, involving windowed averages of KER scores, a check on the residual trend ($b \leq 0$), and the satisfaction of various boolean flags (`corridorsok`, `planesok`, `topologyok`) . While these conditions are tracked in tables like `lanestatusshard` and `lane_thresholds`, they are not aggregated into a single, definitive "admissible" flag. Consequently, an agent cannot simply query the database to determine if a workload meets the criteria for a target lane; it must execute a multi-stage process involving aggregations, trend analysis, and conditional checks. This makes the governance process opaque and difficult to audit. A `vlane_admissibility` view would codify these complex rules into a single, transparent, and reusable database object. This view would take a workload and a target lane as input and return a simple boolean output, effectively turning the governance policy into a machine-verifiable fact stored alongside the data it governs .

Finally, the connection between historical performance, represented by knowledge windows, and forward-looking rewards is not yet solidified into a concrete database artifact. The concept of evaluating a shard's KER over short, medium, and long-term windows is defined, and the principle of using these windows to determine reward eligibility is sketched out . However, there is no view that explicitly connects the results of a windowing function (like `dr9_knowledge_windows_view`) with the corridor thresholds applicable to a given lane . This creates a gap between performance measurement and economic incentives. The proposal for a `vreward_window_lane` view aims to fill this void by providing a direct SQL-based mechanism to apply lane-specific thresholds to the windowed KER scores, thereby determining if a shard is eligible for rewards . This operationalizes the link between sustained performance and compensation, making the incentive structure transparent and queryable. Collectively, these gaps highlight the need for a concerted effort to move from a fragmented, code-dependent safety model to an integrated, data-driven one, where the essential safety and governance logic is expressed as a set of standardized, queryable database surfaces.

Consolidating Shard-Level Safety Metrics: The Lyapunov Residual and KER Views

To address the identified gap between the frozen mathematical invariants and their practical application, the most immediate priority is to create consolidated, shard-centric views that compute and present the core safety metrics. These views will transform the distributed components of the safety model—Lyapunov residual terms, risk coordinates, and KER scores—into a single, coherent, and directly queryable representation for each shard instance. This approach adheres strictly to the existing ecosafety grammar and DR1–DR10 specifications, avoiding the invention of new theory and instead focusing on systematic projection . The development of two primary views, `vshardresidual` and `vshardker`, is central to this effort. The former will serve as the engine for calculating the full Lyapunov residual, V_t, while the latter will act as a comprehensive dashboard, presenting the residual alongside the derived KER scores, lane status, and other metadata. These views will become the foundational layer upon which all subsequent safety assessments, from lane governance to placement advisories, can be built.

The first and most fundamental object to be created is the `vshardresidual` view. Its sole purpose is to compute the Lyapunov residual, V_t, for every entry in the `shardinstance` table. This view will encapsulate the core formula, $V_t = \sum_j w_j r_{t,j}^2$, by performing the necessary aggregations directly within the database . The construction of

this view involves joining the `shardinstance` table—which contains the `shardid` and `timestamp`—with the `shardriskcoord` table, which holds the per-plane risk values ($r_{t,j}$), and the `residualterm` table, which provides the corresponding weights or coefficients (w_j, also labeled `alpha`) . The view will group the results by `shardid` and `timestamp` to calculate a single residual value for each shard at each point in time. The output of this view will include the `shardid`, `timestamp`, and the computed column `Vt`.

Column Name	Data Type	Description
`shardid`	TEXT	Foreign key referencing `shardinstance.shardid`.
`timestamp`	DATETIME	The timestamp for which the residual is calculated, from `shardinstance.timestamp`.
`Vt`	REAL	The computed Lyapunov residual, $V_t = \sum_j w_j r_{t,j}^2$.

By moving this calculation into the database, `vshardresidual` offers several advantages. First, it eliminates the need for application code to re-implement the residual formula, reducing the potential for bugs and ensuring consistency across all agents that query the database. Second, it enables powerful time-series analysis. Analysts can now use standard SQL functions to track the evolution of V_t over time, detect trends, and set up alerts for any violations of the non-increasing invariant (V_{t+1} > V_t). Third, it creates a standardized data product that can be consumed by any downstream service, from a lane governor to a monitoring dashboard, without requiring intimate knowledge of the underlying schema. The creation of this view is a direct response to the identified need for a canonical shard-level residual surface, transforming a distributed calculation into a centralized, queryable asset .

Building upon the foundation of `vshardresidual`, the second and more comprehensive view is `vshardker`. This view will synthesize the Lyapunov residual with the other critical safety metrics to provide a holistic snapshot of a shard's safety posture. It will serve as a "one-stop shop" for any agent needing to assess a shard's eligibility for a lane or its general health. The construction of `vshardker` involves a multi-layered join. It will start with the `vshardresidual` view and join it with the `shardinstance` table to pull in the directly measured KER scores (K_t, E_t, R_t), the `lane` designation, and the `kerdeployable` flag . To add contextual metadata, it will join with the `residualkernel` table (via the `planecontractid` in `shardinstance`) to include the kernel's name and version, providing traceability back to the governing ALN contract . Finally, it will incorporate the topology risk components by joining with the proposed `vshardtopologyker` view, bringing in `Itopology`, `rtopology`, and the final residual including the topology term .

Column Name	Data Type	Description
`shardid`	TEXT	Unique identifier for the shard.
`timestamp`	DATETIME	The time instant of the safety snapshot.
`Vt`	REAL	The core Lyapunov residual, V_t, from `vshardresidual`.
`K`	REAL	The Knowledge score, K_t, from `shardinstance`.
`E`	REAL	The Energy score, E_t, from `shardinstance`.
`R`	REAL	The Risk score, R_t, from `shardinstance`.
`lane`	TEXT	The current operational lane of the shard, from `shardinstance`.
`rtopology`	REAL	The normalized topology risk coordinate, $r_{topology}$, from `vshardtopologyker`.
`Vt_with_topology`	REAL	The final Lyapunov residual including the topology penalty, from `vshardtopologyker`.
`kernel_name`	TEXT	The name of the residual kernel, from `residualkernel`.
`kernel_version`	TEXT	The version of the residual kernel, from `residualkernel`.

The value of `vshardker` is immense. It consolidates the entire front-end of the safety model—the observable metrics—into a single, easy-to-query entity. An agent can now retrieve the complete safety profile of a shard with a simple command: `SELECT * FROM vshardker WHERE shardid = '...';`. This simplicity is deceptive; behind the scenes, the view orchestrates a complex series of joins and calculations that were previously scattered across different layers of the software stack. It makes the relationship between the residual, the KER scores, the lane, and the topology risk explicit and tangible. This view becomes the primary interface for any system that needs to make a safety-related decision about a shard. For instance, a lane governor could query `vshardker` to verify that a shard's residual is below a certain threshold before considering a promotion. A placement advisor could check `rtopology` to ensure that a shard with poor governance integrity is not placed in a sensitive location. By creating these two views, we are not adding new mathematics; we are organizing the existing mathematics into a format that is accessible, consistent, and directly usable by the agents of the EcoNet constellation, fulfilling the core mandate of the research goal .

Integrating Governance Drift: The Role of Topology Risk in the Lyapunov Function

A critical enhancement to the ecosafety framework is the formal integration of governance drift as a quantifiable risk factor within the Lyapunov stability model. This is

achieved through the `rtopology` metric, which transforms abstract concepts of manifest completeness and role-labeling consistency into a concrete, numerical value that directly impacts the system's perceived stability . While the calculation of the inconsistency index ($I_{topology}$) and its normalization into $r_{topology}$ are specified, and its injection into the residual is defined, the current implementation lacks a unified surface that makes this relationship clear and queryable . To rectify this, the proposed `vshardtopologyker` view will be developed. This view will serve as the central nexus where the raw output of the topology audit is fused with the core Lyapunov residual calculation, providing a complete and auditable record of how governance health influences a shard's overall safety score. This object is essential for surfacing the consequences of governance drift and ensuring that maintaining a well-governed network is intrinsically linked to achieving a low Lyapunov residual.

The `vshardtopologyker` view will be constructed by augmenting the `vshardker` view with the specific components of the topology risk calculation. Its primary inputs will be the `vshardker` view, which provides the core residual (V_t) and associated shard metadata, and the `topologyriskmetric` table, which contains the latest audit results for each shard, including the raw inconsistency index ($I_{topology}$) and the derived normalized coordinate ($r_{topology}$) . Additionally, it will need to join with the `planeweights` table to retrieve the specific weight, w_{topology}, assigned to the topology plane for the shard's governing `planecontractid` . The key calculation within this view will be the computation of the final, augmented Lyapunov residual. This will be done by taking the core residual from `vshardker` and adding the penalty term contributed by topology risk: $V_t^{\text{final}} = V_t^{\text{core}} + w_{\text{topology}} r_{\text{topology}}^2$. This formula is a direct encoding of the frozen math, ensuring that the database mirrors the exact logic used by the application-level `checksafestep` function .

The output of the `vshardtopologyker` view will extend the `vshardker` output by adding the raw and normalized topology metrics, the topology weight, and the final residual. This provides a transparent, step-by-step breakdown of the residual's composition.

Column Name	Data Type	Description
`shardid`	TEXT	Unique identifier for the shard.
`timestamp`	DATETIME	The time instant of the safety snapshot.
`Vt`	REAL	The core Lyapunov residual, V_t, from `vshardker`.
`K`	REAL	The Knowledge score, K_t.
`E`	REAL	The Energy score, E_t.
`R`	REAL	The Risk score, R_t.
`lane`	TEXT	The current operational lane of the shard.
`rtopology`	REAL	The normalized topology risk coordinate, r_{topology}, from `topologyriskmetric`.
`Itopology`	INTEGER	The raw topology inconsistency index, I_{topology}, from `topologyriskmetric`.
`w_topology`	REAL	The weight for the topology plane, from `planeweights`.
`Vt_with_topology`	REAL	The final Lyapunov residual including the topology penalty: $V_t + (w_{\text{topology}} \times r_{\text{topology}}^2)$.
`kernel_name`	TEXT	The name of the residual kernel.
`kernel_version`	TEXT	The version of the residual kernel.

This detailed output is transformative for governance and auditing. First, it makes the penalty for governance drift explicit and measurable. An administrator can now see exactly how many unlabeled repositories or missing manifests have pushed a shard's residual over a critical threshold. This turns a passive audit log into an active safety control. Second, it allows for the enforcement of corridor constraints on the `rtopology` coordinate itself. The `corridordefinition` table can define safe, gold, and hard bands for `rtopology`, just like it does for other risk coordinates . An agent can then query `vshardtopologyker` to find all shards where `rtopology` has entered the gold or hard band, triggering corrective actions or quarantine procedures. Third, by integrating this logic into a database view, the enforcement of the non-compensation rule related to topology becomes more robust. While the core logic resides in the `checksafestep` function, the database now provides a clear and auditable trail of the variables involved in that decision. The `vshardtopologyker` view ensures that the data fed into that function is consistent, complete, and directly traceable. Ultimately, this object operationalizes the principle that a healthy network structure is a prerequisite for a stable system, embedding the cost of governance failure directly into the language of stability.

Modeling Spatial and Temporal Reach: Blast Radius and Canal Velocity Views

Beyond the internal stability of individual shards, the safety of the EcoNet constellation depends critically on the spatial and temporal relationships between its components. Failures or anomalies in one node can propagate through the network, and the scope of this potential impact is defined by the blast radius of each node. The concept of a "canal" further describes directed pathways of influence propagation. To make these spatial dynamics queryable, two new views are proposed: `vshardblast` and `vshardcanal`. The `vshardblast` view will consolidate the dispersed information about blast radius into a standardized, per-shard summary, while `vshardcanal` will build upon this to compute a derived metric, "canal velocity," and expose it as a first-class risk coordinate. These views are not introducing new physics or new safety rules; they are systematically projecting the existing definitions from the ecosafety grammar and DR3 into concrete database artifacts, allowing agents to reason about physical proximity and propagation speed directly through SQL .

The `vshardblast` view will provide a comprehensive overview of a shard's blast radius characteristics. Its construction requires joining the `shardinstance` table with the `blastradiusobject` table. This mapping could be direct if a shard is tied to a single blast radius definition, or it could involve a more complex mapping table if different parts of a shard have different radii. The view will expose all the key attributes that define a blast radius, providing a single, authoritative source for this critical piece of spatial metadata. The inclusion of the `tbr2026v1hex` descriptor, a compact hex-encoded template, ensures that agents can easily interpret and validate blast radius definitions consistently .

Column Name	Data Type	Description
`shardid`	TEXT	Foreign key referencing `shardinstance.shardid`.
`region`	TEXT	Geographic or logical region where the shard resides.
`nodeid`	TEXT	Identifier for the node hosting the shard.
`radius_meters`	REAL	Maximum physical distance of the blast radius.
`radius_hours`	REAL	Maximum propagation time of the blast radius.
`hops`	INTEGER	Maximum graph hop count of the blast radius.
`primary_planes`	TEXT	Comma-separated list of primary risk planes affected.
`secondary_planes`	TEXT	Comma-separated list of secondary risk planes affected.
`continuity_grade`	TEXT	The grade of continuity required for this node's placement.
`tbr2026v1hex`	TEXT	Hexadecimal descriptor of the blast radius object.

The utility of `vshardblast` is profound for system operators and placement advisors. With this view, an advisor can execute a query to find all pairs of nodes whose blast radii overlap, preventing the unsafe co-location of high-risk components. For example, a query could identify any two shards in the `Phoenix-AZ` region where the `distance_meters` between them is less than the sum of their respective `radius_meters`. This moves the responsibility for collision detection from ad-hoc application logic to the database, ensuring correctness and efficiency. Furthermore, by making the blast radius parameters queryable, it becomes possible to automate checks against standards like ISO 13857:2019 safety distances, which are designed to prevent hazard zones from being reached by adjacent equipment or personnel . The `vshardblast` view thus serves as the foundational layer for all spatial safety reasoning within the EcoNet.

Building upon the data provided by `vshardblast`, the `vshardcanal` view will introduce a dynamic element into the safety calculus by computing a "canal velocity." This concept, inspired by ideas in urban planning and infrastructure networks, models the speed at which an influence (e.g., a contamination event or a software update) can travel along the adjacency graph . The velocity is calculated as the ratio of the spatial extent of the blast radius to its temporal extent:
\text{velocity} = \frac{\text{radius_meters}}{\text{max(radius_hours, epsilon})}
where epsilon is a small constant to prevent division by zero . This yields a metric in meters per hour. More importantly, this raw velocity can be normalized into a dimensionless risk coordinate, r_{canal}, by dividing it by a characteristic maximum velocity (v_{max}) for the given region or plane, as defined by a corridor in the `corridordefinition` table. The resulting value, clamped to the range $[0,1]$, represents the degree to which the canal's propagation speed exceeds its safe limit .

Column Name	Data Type	Description
`shardid`	TEXT	Unique identifier for the shard.
`region`	TEXT	Geographic or logical region where the shard resides.
`nodeid`	TEXT	Identifier for the node hosting the shard.
`radius_meters`	REAL	From `vshardblast`.
`radius_hours`	REAL	From `vshardblast`.
`canal_velocity_m_per_h`	REAL	Calculated as `radius_meters / NULLIF(radius_hours, 0)`.
`r_canal`	REAL	Normalized canal velocity risk coordinate, clamped to [0,1].
`vmax_m_per_h`	REAL	The maximum allowed canal velocity from a corridor definition.

By exposing r_{canal} as a risk coordinate, the `vshardcanal` view integrates the concept of propagation speed seamlessly into the existing Lyapunov residual framework. The `corridordefinition` table can define safe/gold/hard bands for `r_canal`, and the `residualterm` table can assign it a weight, w_{canal} . This means that a shard operating in a narrow, congested "canal" with a high propagation velocity will contribute more to the Lyapunov residual than one with a slower velocity, just like a shard with high energy consumption or poor topology. This elevates canal dynamics from a qualitative description to a quantitative safety constraint, allowing the system to actively discourage configurations that create fast-propagating failure modes. The creation of these two views completes the cycle of operationalizing the frozen math: the concepts of blast radius and canal are now concrete, queryable, and integral parts of the EcoNet's safety model.

Codifying Governance Rules: Lane Admissibility and Reward Corridors

The final and perhaps most impactful step in operationalizing the ecosafety framework is to codify the abstract governance rules for lane progression and reward distribution into explicit, queryable database objects. The rules for moving an artifact from a `RESEARCH` lane to a `PROD` lane are complex, involving not only instantaneous KER scores but also their behavior over time, adherence to corridor constraints, and the status of non-offsettable planes . Similarly, the principle of rewarding sustained performance across different time horizons must be translated into a concrete mechanism. The proposed views, `vlane_admissibility` and `vreward_window_lane`, are designed to fulfill this purpose. They will transform the prose descriptions and Rust sketches of governance policies into standardized database surfaces, making the rules discoverable, auditable,

and enforceable directly through SQL. This approach ensures that governance becomes a structural property of the EcoNet's data fabric, not just a runtime constraint enforced by application logic .

The `vlane_admissibility` view will be the central arbiter of lane transitions. It will take a workload and a target lane as inputs and return a boolean flag, `is_admissible`, indicating whether the transition is permitted. The construction of this view will be a synthesis of multiple data sources. It will start with the `shardinstance` table and join it with a pre-calculated view, tentatively named `vkernel_lane_window`, which computes the required windowed averages of K, E, and R for the specified time periods (short, medium, long) . It will also join with the `lanestatusshard` table to get the status of various predicates like `corridorsok`, `planesok`, and `topologyok` . The core logic will be a series of case statements or boolean expressions that check if all the conditions for the target lane are met. These conditions include: 1. **KER Thresholds:** The windowed K, E, and R scores must all meet or exceed the minimum thresholds defined for the target lane in the `lanepolicy` table. 2. **Residual Trend:** The trend of the Lyapunov residual, `Vtrend_b`, must be non-positive, satisfying the Lyapunov descent requirement . 3. **Predicate Flags:** All relevant boolean flags from `lanestatusshard` must be true.

Furthermore, to align with the DR7 lane quarantine concept, the view will incorporate a `lane_quarantined` flag, which would override the admissibility check if a shard is under investigation or awaiting review . The output of this view will provide a definitive answer to the question, "Can this workload enter this lane?" and a detailed breakdown of the checks performed.

Column Name	Data Type	Description
`workload_id`	TEXT	The unique identifier for the workload being evaluated.
`target_lane`	TEXT	The lane to which the workload is attempting to transition.
`is_admissible`	BOOLEAN	TRUE if all admissibility criteria are met, FALSE otherwise.
`K_window`	REAL	Windowed average K score.
`E_window`	REAL	Windowed average E score.
`R_window`	REAL	Windowed average R score.
`Vtrend_b`	REAL	Slope of the residual trend line.
`corridors_ok`	BOOLEAN	Status from `lanestatusshard.corridorsok`.
`planesok`	BOOLEAN	Status from `lanestatusshard.planesok`.
`topologyok`	BOOLEAN	Status from `lanestatusshard.topologyok`.
`lane_quarantined`	BOOLEAN	TRUE if the workload is in quarantine.

This view makes the governance process transparent and machine-verifiable. A continuous integration pipeline could call this view before deploying a new version of a shard, failing the build if the result is `FALSE`. This embeds safety checks directly into the development lifecycle. It also provides a clear audit trail, as the intermediate values (`K_window`, `Vtrend_b`, etc.) are all part of the query's output, allowing anyone to inspect the reasoning behind a governance decision. The creation of `vlane_admissibility` is a crucial step toward turning the ecosafety framework from a set of guidelines into a hard, enforceable contract.

Parallel to lane governance, the `vreward_window_lane` view will operationalize the connection between historical performance and economic incentives. This view builds upon the `vshard_ker_knowledge` view, which joins `shardinstance` with `knowledgeecoscore` and the `dr9_knowledge_windows_view` to expose KER scores averaged over short, medium, and long windows . The purpose of `vreward_window_lane` is to apply the corridor thresholds specific to a given lane to these windowed scores to determine reward eligibility. For each shard, the view will calculate a boolean flag for each window (e.g., `short_eligible`, `medium_eligible`, `long_eligible`) based on whether the windowed KER scores fall within the "gold" or "hard" bands defined for the shard's current lane in the `corridordefinition` table. Finally, it will compute a master `reward_eligible` flag, which could be implemented as the logical AND (or "minimum") of the individual window flags, reflecting the principle of requiring sustained performance across all time horizons .

Column Name	Data Type	Description
`shardid`	TEXT	The unique identifier for the shard.
`short_eligible`	BOOLEAN	Eligible based on the short-term window.
`medium_eligible`	BOOLEAN	Eligible based on the medium-term window.
`long_eligible`	BOOLEAN	Eligible based on the long-term window.
`reward_eligible`	BOOLEAN	Eligible for reward if all window conditions are met.

This view provides a direct, SQL-based implementation of the reward mechanism. It cleanly separates the calculation of performance windows from the application of corridor thresholds, making the system modular and easy to adapt. If the thresholds for a lane change, only the lookup in `corridordefinition` needs to be updated; the logic within the view remains the same. By making the criteria for receiving rewards explicit and queryable, the system fosters a culture of sustained excellence. Agents can query `vreward_window_lane` to understand not just their current standing but also the historical performance required to achieve future rewards. In concert, `vlane_admissibility` and `vreward_window_lane` represent the culmination of

this research effort: they are the tools that translate the frozen mathematical invariants and governance rules into a living, breathing, and self-enforcing system, ready for deployment across the entire EcoNet constellation.

Reference

1. A Sharding Scheme for Large-Scale Internet of Things Application https://dl.acm.org/doi/10.1145/3641290
2. [PDF] Optimal Sharding for Scalable Blockchains with Deconstructed SMR https://arxiv.org/pdf/2406.08252
3. (PDF) Enhancing Blockchain Interoperability and Intraoperability ... https://www.researchgate.net/publication/376489036_Enhancing_Blockchain_Interoperability_and_Intraoperability_Capabilities_in_Collaborative_Enterprise-A_Standardized_Architecture_Perspective
4. Symmetry, Volume 17, Issue 9 (September 2025) – 215 articles - MDPI https://www.mdpi.com/2073-8994/17/9
5. Arxiv今日论文| 2026-02-12 - 闲记算法 http://lonepatient.top/2026/02/12/arxiv_papers_2026-02-12
6. All Topics - cse, hkust https://cse.hkust.edu.hk/~lzhang/topic/Nips/TopicsTable.html
7. 天体物理学2025_3_18 - arXiv每日学术速递 http://arxivdaily.com/thread/65292
8. Improvement and Optimization of Vulnerability Detection Methods ... https://ieeexplore.ieee.org/iel7/6287639/6514899/10192903.pdf
9. Ethereum Smart Contracts: Analysis and Statistics of their Source ... https://www.researchgate.net/publication/339880854_Ethereum_Smart_Contracts_Analysis_and_Statistics_of_their_Source_Code_and_Opcodes
10. Untitled - Springer Nature https://link.springer.com/content/pdf/10.1007/978-4-431-68189-2.pdf
11. Improvement and Optimization of Vulnerability Detection Methods ... https://ieeexplore.ieee.org/iel7/6287639/10005208/10192903.pdf
12. SQLite Error foreign key mismatch - Stock referencing Variants https://stackoverflow.com/questions/55451893/sqlite-error-foreign-key-mismatch-stock-referencing-variants

13. How do i get information about foreign keys in SQLite? https://stackoverflow.com/questions/48508140/how-do-i-get-information-about-foreign-keys-in-sqlite